Student Record Search System Using Binary Search Tree (BST)

Mini Project Report

Name- Deepshikha

Branch- BTech AIDS

Roll No.- 2556580

Introduction: A Student Record Search System is a small program that stores student details and lets a user add, find, remove and display records. In this project, a Binary Search Tree (BST) is used to organize records according to each student's unique enrolment ID.

A BST is a binary tree where smaller keys are placed in the left subtree and larger keys are placed in the right subtree. This rule helps the program search for a record without checking every record in many cases.

Objectives:

1. To understand how a BST stores data.
2. To insert, search and delete student records by enrolment ID.
3. To prevent duplicate enrolment IDs.
4. To display records in sorted order and perform three tree traversals.
5. To compare the effect of insertion order on search comparisons.

Data Structure Design: Each student record contains three fields: enrolment ID, student name and department. Each tree node stores one student record and two links: a left-child pointer. The enrolment ID is the key used to place records in the tree.

If a new ID is smaller than the current node's ID, the program moves left. If it is larger, the program moves right. If it is equal, the ID is already present and the new record is rejected.

Main Operations and Algorithms:

1. **Insertion:** Start at the root. If the current position is empty, create a new node. Otherwise compare the new enrolment ID with the current ID. Move left for a smaller ID and right for a larger ID. If the ID matches an existing record, report a duplicate and do not insert it.
2. **Searching:** Compare the required ID with the current node. If they match, the record is found. If the required ID is smaller, continue in the left subtree; otherwise continue in the right subtree. If the pointer becomes NULL, the record does not exist.
3. **Deletion:** Deletion has three cases. (i)Leaf node: remove the node directly. (ii)One child: connect the parent to the node's only child. (iii)Two children: find the smallest node in the right subtree, copy that record into the node being deleted, and then remove the successor from its old position.
4. **Tree Traversals:** Inorder: Left, Root, Right. In a BST, this displays records in ascending ID order. Preorder: Root, Left, Right. Postorder: Left, Right, Root.

Sample Records:

ID	Student Name	Department
50	Aman	CSE
30	Priya	AIDS
70	Rahul	CSE
20	Neha	ECE
40	Simran	AIDS
60	Karan	CSE
80	Riya	ECE
10	Mohit	CSE
35	Anjali	AIDS
65	Arjun	ECE

Testing Plan:

Test	Action	Expected Result
1	Search ID 40	Record for Simran is found
2	Search ID 99	Record not found
3	Insert an existing ID, such as 50	Duplicate ID rejected
4	Delete ID 10	Leaf node deletion succeeds
5	Delete ID 60	One-child deletion succeeds because 1060 has child 1065
6	Delete ID 30	Two-child deletion succeeds
7	Inorder traversal	IDs appear in ascending order

Efficiency Comparison: The program searches IDs 1 through 10 in two BSTs. The first uses a balanced-looking insertion order (5,3,8,2,4,7,9,1,6,10), while the second inserts IDs in ascending order (1 through 10). The second order creates a skewed tree, so it searches generally require node checks.

For the specified sequences, the balanced-looking tree has height 4 and 29 total search comparisons (average 5.50). These are the results for these particular sequences, not a guarantee for every BST.

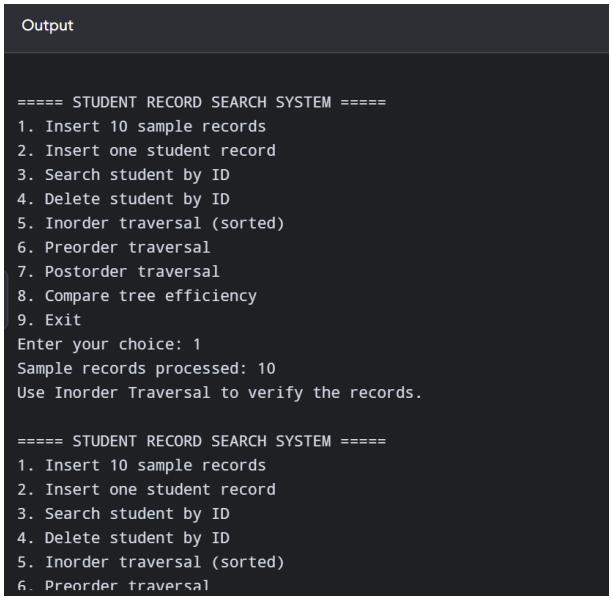
Time and Space Complexity:

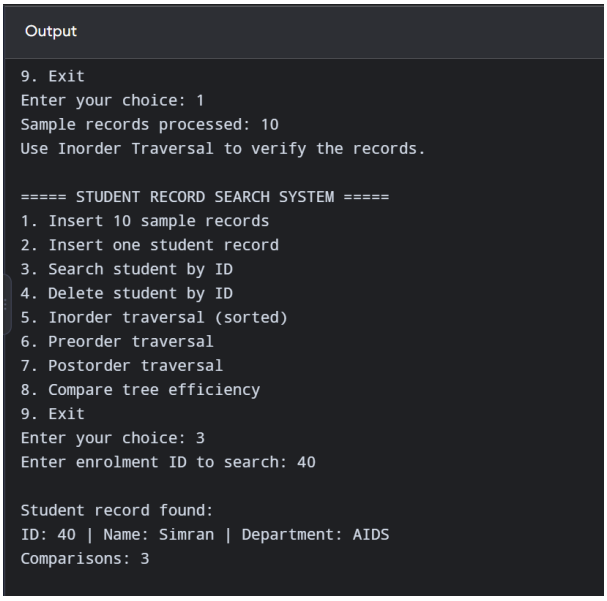
Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Searching	O(log n)	O(n)
Deletion	O(log n)	O(n)
Traversal	O(n)	O(n)

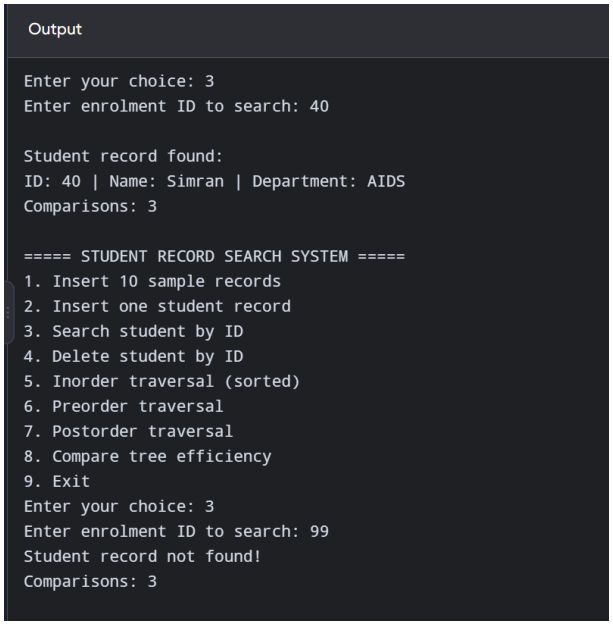
AVL Tree: An AVL tree is a self-balancing Binary Search Tree. For every node, its balance factor is height(left subtree) minus height(right subtree). The balance factor must be -1, 0, +1. If insertion or deletion makes a node unbalanced, rotations (LL, RR, LR or RL) restore balance. An AVL tree keeps search, insertion and deletion at O(log n) time. The current project uses a normal BST and does not implement AVL rotations.

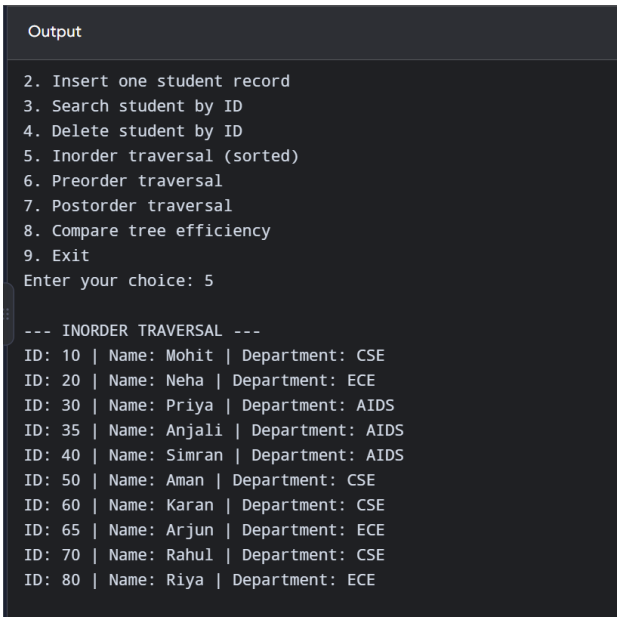
Conclusion: This project demonstrates how a Binary Search Tree can be used to manage student records. It covers insertion, searching, deletion, duplicate handling and traversals. The comparison feature also shows why the order in which data is inserted can affect the efficiency of a normal BST.

Screenshots of the outputs:









Output

```
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 6

--- PREORDER TRAVERSAL ---
ID: 50 | Name: Aman | Department: CSE
ID: 30 | Name: Priya | Department: AIDS
ID: 20 | Name: Neha | Department: ECE
ID: 10 | Name: Mohit | Department: CSE
ID: 40 | Name: Simran | Department: AIDS
ID: 35 | Name: Anjali | Department: AIDS
ID: 70 | Name: Rahul | Department: CSE
ID: 60 | Name: Karan | Department: CSE
ID: 65 | Name: Arjun | Department: ECE
ID: 80 | Name: Riya | Department: ECE
```

Output

```
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 7

--- POSTORDER TRAVERSAL ---
ID: 10 | Name: Mohit | Department: CSE
ID: 20 | Name: Neha | Department: ECE
ID: 35 | Name: Anjali | Department: AIDS
ID: 40 | Name: Simran | Department: AIDS
ID: 30 | Name: Priya | Department: AIDS
ID: 65 | Name: Arjun | Department: ECE
ID: 60 | Name: Karan | Department: CSE
ID: 80 | Name: Riya | Department: ECE
ID: 70 | Name: Rahul | Department: CSE
ID: 50 | Name: Aman | Department: CSE
```

Output

```
===== STUDENT RECORD SEARCH SYSTEM =====
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 4
Enter enrolment ID to delete: 10
Record deleted successfully.

===== STUDENT RECORD SEARCH SYSTEM =====
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
```

Output

```
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 4
Enter enrolment ID to delete: 10
Record deleted successfully.

===== STUDENT RECORD SEARCH SYSTEM =====
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 4
Enter enrolment ID to delete: 60
Record deleted successfully.
```

Output

```
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 4
Enter enrolment ID to delete: 60
Record deleted successfully.

===== STUDENT RECORD SEARCH SYSTEM =====
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 4
Enter enrolment ID to delete: 30
Record deleted successfully.
```

Output

```
1. Insert 10 sample records
2. Insert one student record
3. Search student by ID
4. Delete student by ID
5. Inorder traversal (sorted)
6. Preorder traversal
7. Postorder traversal
8. Compare tree efficiency
9. Exit
Enter your choice: 8

--- TREE EFFICIENCY COMPARISON ---

Balanced-looking BST:
Height: 4 nodes
Total search comparisons: 29
Average comparisons: 2.90

Sorted-insertion BST:
Height: 10 nodes
Total search comparisons: 55
Average comparisons: 5.50
```